

AI Job Hunt Agent

Complete Beginner Learning Guide and Technical Manual

Project path: /Users/akbar/job-hunt-agent

Focus: concepts, architecture, implementation, APIs, AI/ML, best practices, interviews, and exercises

Table of Contents

Section	Focus
1. Project Overview	Concepts, implementation, exercises, and interview prep
2. Final Product Flow	Concepts, implementation, exercises, and interview prep
3. Architecture, Data Flow, API Flow, Model Flow, Database Flow, Deployment Flow	Concepts, implementation, exercises, and interview prep
4. File-by-File Guide	Concepts, implementation, exercises, and interview prep
5. Library, API, and Component Guide	Concepts, implementation, exercises, and interview prep
6. AI and ML Concepts	Concepts, implementation, exercises, and interview prep
7. Important Code Walkthroughs	Concepts, implementation, exercises, and interview prep
8. Best Practices and Common Mistakes	Concepts, implementation, exercises, and interview prep
9. Portfolio Presentation Guide	Concepts, implementation, exercises, and interview prep
10. Interview Questions and Practice Answers	Concepts, implementation, exercises, and interview prep
11. Full Source Snapshot	Concepts, implementation, exercises, and interview prep

1. Project Overview

This manual explains your AI Job Hunt Agent project as if you are learning it from zero. The goal is not only to run the app, but to understand why each technology exists, how the files connect, and how to improve the project into a strong portfolio piece.

The product idea is simple: upload a resume, search jobs, compare each job against the resume, score the fit, explain the gaps, and produce application help such as a cover letter draft.

Current project name: AI Job Search Agent.

Recommended portfolio name: AI Job Hunt Copilot.

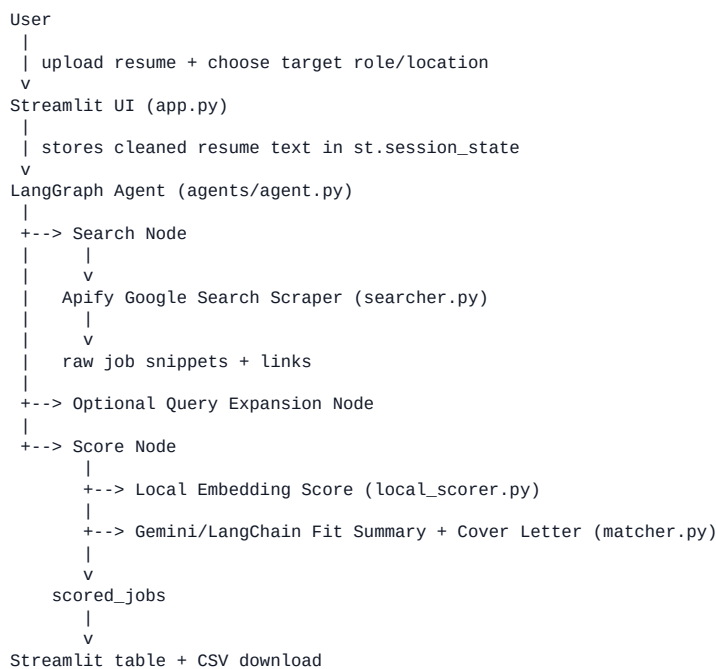
What problem does it solve?

Job seekers often do not know which jobs are worth applying to. Job descriptions are long, resumes are hard to tailor, and matching skills manually is slow. This project automates the first pass: it reads a resume, searches roles, scores matches, and explains why a job is or is not a good fit.

What makes it portfolio-worthy?

- It combines frontend, APIs, AI, embeddings, workflow orchestration, and data export.
- It solves a real user problem that recruiters and job seekers understand.
- It shows practical AI engineering instead of only calling an LLM once.
- It can grow into RAG, vector search, tracking, analytics, and deployment.

Current high-level architecture



Common mistakes

- Building only a script and not a clear user workflow.
- Showing an AI answer without explaining the reasoning or evidence.
- Ignoring current code gaps when presenting the project.

Exercises

- Explain the product in 30 seconds as if you are talking to an interviewer.
- Write three user stories: one for a student, one for an experienced developer, and one for a recruiter.
- Draw the architecture from memory and compare it with the diagram.

Interview questions

- What problem does your app solve?
- Why did you use both embeddings and an LLM?
- What is the difference between a demo app and a production-ready app?

2. Final Product Flow

The best final flow for the portfolio is:

Upload Resume -> Extract Resume Profile -> Confirm Search Target -> Search Jobs -> Fetch or collect Job Descriptions -> Score Each Job -> Rank Jobs -> Show Gap Report -> Generate Cover Letter.

Your current project already has parts of this, but some parts are not yet fully connected.

Ideal user flow

PDF resume

- > extract text with pypdf
- > clean whitespace
- > keep resume text in Streamlit session state
- > user enters target job title and location
- > Apify search returns job links and snippets
- > each snippet becomes the temporary job description
- > SentenceTransformer converts resume and snippet into vectors
- > cosine similarity creates Match Score
- > Gemini generates Fit Summary and Cover Letter Draft
- > Pandas converts scored jobs into a CSV table
- > Streamlit displays and downloads results

Current implemented flow

- User uploads a PDF resume in Streamlit.
- App stores cleaned resume text in Streamlit session state.
- User manually enters target role and location.
- LangGraph agent searches jobs through Apify.
- Agent scores each job using local embeddings.
- Agent asks Gemini through LangChain for fit summary and cover letter.
- Streamlit displays scored jobs and offers a CSV download.

Missing or incomplete flow pieces

- The app imports `utils/pdf_reader.py`, but that file is missing in the current workspace. The compiled pycache exists, but source should be restored.
- `extract_resume_details()` exists but is not connected to `app.py`, so the resume profile is not shown or used to infer target roles.
- The app still asks the user to type a role instead of suggesting roles from the resume.
- Job search currently uses search result snippets, not full job descriptions. This limits scoring accuracy.
- There is no real database yet. `jobs_database.csv` acts as simple file storage.
- The agent checks only the exact string `API_ERROR`, while `searcher.py` returns strings beginning with `ERROR:`. This can cause incorrect state handling.
- There is no deployed URL, README, screenshots, or tests yet.

Exercises

- Write the ideal flow as a numbered list in your own words.
- Identify which current file owns each step in the flow.
- Create a checklist of the next five implementation tasks.

Interview questions

- What is the main bottleneck in your current job matching quality?
- Why is fetching full job descriptions better than using snippets?
- How would you handle invalid or scanned PDF resumes?

3. Architecture, Data Flow, API Flow, Model Flow, Database Flow, Deployment Flow

Architecture means how the parts are arranged. Data flow means how information moves. API flow means what external services are called. Model flow means how the AI models receive input and return output. Database flow means how records are stored and retrieved. Deployment flow means how the app becomes usable outside your laptop.

Architecture flow

Streamlit is the user interface. app.py collects inputs and starts the graph. agents/agent.py controls the multi-step workflow. searcher.py talks to Apify. local_scorer.py computes embedding similarity. matcher.py calls Gemini through LangChain. searcher.py also saves final results into CSV.

API flow

The app reads API keys from .env using python-dotenv. The Apify key is used by ApifyClient to call the google-search-scraper actor. The Gemini key is used by LangChain's ChatGoogleGenerativeAI wrapper. Data leaves your app as search queries, resume text, and job snippets, then returns as search results and AI-generated JSON.

Model inference flow

```
resume text + job description/snippet
|
+--> local_scorer.py
|       text -> embedding vectors -> cosine similarity -> numeric score
|
+--> matcher.py
|       prompt template -> Gemini model -> JSON parser -> fit summary + cover letter
```

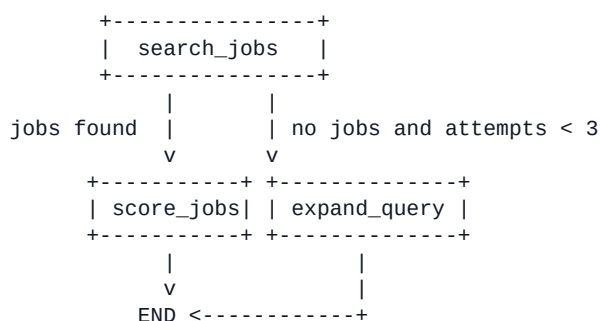
Database flow

The current app does not use a true database. It uses Pandas to write jobs_database.csv. CSV is good for learning and demos, but a real app should use SQLite, PostgreSQL, or Supabase so jobs, users, statuses, and match reports can be queried safely.

Deployment flow

A simple deployment path is Streamlit Community Cloud or Render. A stronger portfolio path is: create requirements.txt, restore missing source files, add tests, hide secrets in environment variables, deploy Streamlit, add screenshots and demo video, and write a README. If you later move to a full-stack app, Next.js plus FastAPI is also a good architecture.

Agent graph flow



Common mistakes

- Committing .env files to GitHub.
- Assuming CSV is enough for multi-user production storage.

- Sending too much private resume data to APIs without user consent.

Exercises

- Explain the difference between data flow and API flow.
- Replace CSV storage in a design diagram with SQLite tables.
- Write a deployment checklist for Streamlit Cloud.

Interview questions

- Where do secrets live in this project?
- What data is sent to Gemini?
- How would you add persistent job tracking?

4. File-by-File Guide

This section explains each project file: what it is, why it exists, real-world use cases, key functions/classes, important line explanations, alternatives, exercises, and interview angles.

app.py - Streamlit user interface and workflow launcher

What it is: app.py is the main entry point. Streamlit runs this file and turns Python statements into a web interface.

Why used here: It lets you build a data/AI app quickly without writing HTML, CSS, and JavaScript.

Real-world use cases: AI demos, dashboards, internal tools, ML model prototypes, data analysis apps.

Key components:

- load_dotenv() loads keys from .env.
- st.file_uploader() accepts the PDF resume.
- st.session_state stores resume text between interactions.
- st.text_input() collects target role and location.
- job_agent_app.invoke(initial_state) starts the LangGraph workflow.
- st.dataframe() displays scored jobs.
- st.download_button() returns the CSV.

Important lines:

- Lines 1-6 import environment, UI, Pandas, OS, time, and the compiled agent.
- Lines 13-15 read GOOGLE_API_KEY and APIFY_API_KEY from .env.
- Lines 25-27 initialize session memory for cleaned resume text.
- Lines 34-42 save uploaded PDF, extract text, clean it, and keep it in memory.
- Lines 59-67 validate keys, role, and resume before starting the agent.
- Lines 71-80 build the agent state dictionary.
- Line 84 invokes the graph.
- Lines 91-113 display and download final results.

Current gap: app.py imports utils.pdf_reader, but utils/pdf_reader.py is missing in the current source tree.

Alternative UI tools:

- Gradio: faster for ML demos with simple input/output.
- Flask/FastAPI plus React: more flexible, more work.
- Dash: good for analytics dashboards.

```
# Minimal Streamlit pattern
import streamlit as st

st.title("AI Job Hunt Copilot")
resume = st.file_uploader("Upload resume", type=["pdf"])
role = st.text_input("Target role")

if st.button("Analyze"):
    st.write("Start the workflow here")
```

agents/agent.py - LangGraph workflow controller

What it is: agents/agent.py defines a stateful graph. The graph decides what step runs next.

Why used here: A job hunt workflow has multiple steps: search, maybe retry with a broader query, score jobs, then finish. LangGraph is useful when AI apps need controlled multi-step behavior instead of one function call.

Real-world use cases: research agents, customer support workflows, multi-step data pipelines, tool-using AI assistants.

Key components:

- JobAgentState: the memory schema for the graph.
- search_for_jobs(): calls the search API.

- `expand_search_query()`: broadens the role when no jobs are found.
- `score_found_jobs()`: adds match score, fit summary, and cover letter.
- `route_after_search()`: decides the next node.
- `StateGraph`: builds the graph.
- `END`: marks graph completion.

Important lines:

- Lines 10-19 define the state fields carried through the workflow.
- Lines 23-41 search jobs and update `raw_jobs`.
- Lines 43-52 expand the search query.
- Lines 54-90 score each job.
- Lines 93-106 route based on results and attempts.
- Lines 108-137 build and compile the graph.

Current gap: `search_for_jobs` checks `jobs != "API_ERROR"`, but `searcher.py` returns `"ERROR: ..."`. Use a consistent error type in future code.

Alternative workflow tools:

- Plain Python functions: enough for simple linear flows.
- CrewAI: useful for multi-agent demos, sometimes heavier.
- AutoGen: good for conversation-style agents.
- Airflow/Prefect: better for scheduled production data pipelines.

```
from langgraph.graph import StateGraph, END
```

```
workflow = StateGraph(dict)
workflow.add_node("step_one", lambda state: state)
workflow.set_entry_point("step_one")
workflow.add_edge("step_one", END)
app = workflow.compile()
```

searcher.py - Apify job search and CSV export

What it is: `searcher.py` handles job search through Apify and exports job data to CSV.

Why used here: Job boards are hard to scrape directly. Apify gives hosted scraping actors, so your app can request search results without maintaining a custom scraper.

Real-world use cases: web scraping, competitor monitoring, job listing collection, market research.

Key functions:

- `find_live_jobs(job_title, location, apify_api_key, max_results)`: calls Apify.
- `save_to_spreadsheet(jobs_data, filename)`: stores results as CSV and returns downloadable bytes.

Important lines:

- Lines 12-16 build the Google search query.
- Line 18 creates an `ApifyClient`.
- Lines 20-24 define actor input.
- Line 30 runs `apify/google-search-scraper`.
- Lines 33-37 safely read `defaultDatasetId`.
- Lines 45-59 collect organic search result links and snippets.
- Lines 67-97 save and deduplicate CSV rows.

Current gap: search results are snippets, not full job descriptions. For better matching, add a job detail fetcher that opens each job link and extracts the real description.

Alternative search/data tools:

- SerpAPI: paid search API with structured results.
- Tavily: search API designed for AI apps.
- Bright Data: enterprise scraping.

- Direct job APIs: best when available, but many job boards restrict access.

```
from apify_client import ApifyClient

client = ApifyClient("APIFY_TOKEN")
run = client.actor("apify/google-search-scraper").call(
    run_input={"queries": "Python developer remote", "resultsPerPage": 5}
)
```

matcher.py - LLM fit summary and cover letter generator

What it is: matcher.py sends resume text and job text to Gemini through LangChain and parses the result as JSON.

Why used here: Numeric similarity alone cannot explain why a job fits. The LLM creates human-readable reasoning and a cover letter draft.

Real-world use cases: resume feedback, sales email generation, customer support summaries, document comparison.

Key components:

- ChatGoogleGenerativeAI: LangChain wrapper for Gemini.
- PromptTemplate: reusable prompt with variables.
- JsonOutputParser: asks model output to follow a JSON schema.
- JobMatchReport: Pydantic schema for fit_analysis and cover_letter.
- chain = prompt | llm | parser: LangChain expression language pipeline.

Important lines:

- Lines 8-10 define the output fields.
- Lines 15-19 initialize Gemini.
- Line 23 creates the JSON parser.
- Lines 26-41 define prompt text and formatting instructions.
- Line 46 chains prompt, LLM, and parser.
- Line 51 invokes the chain.
- Lines 55-60 wrap returned data in a small Report class.

Alternative model libraries:

- Direct google-genai SDK: less abstraction, more direct control.
- OpenAI SDK: alternative LLM provider.
- Instructor: great for structured outputs with Pydantic.
- Haystack: strong for retrieval pipelines.

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate(
    template="Compare resume {resume} with job {job}. {format_instructions}",
    input_variables=["resume", "job"],
)
```

local_scorer.py - Embeddings and cosine similarity

What it is: local_scorer.py computes a local mathematical match score using sentence embeddings.

Why used here: It gives a fast, cheap score without calling an LLM for the numeric part. This is useful because scoring many jobs with an LLM can be slow and costly.

Real-world use cases: semantic search, recommendation systems, duplicate detection, resume-to-job matching, FAQ matching.

Key components:

- SentenceTransformer('all-MiniLM-L6-v2'): converts text to vectors.
- model.encode(): creates embeddings.
- cosine_similarity(): compares vector direction.
- final_score: percentage-like score shown to user.

Important lines:

- Lines 1-3 import embedding and math libraries.
- Line 7 loads the embedding model.
- Lines 17-22 encode resume and job text, then reshape vectors.
- Lines 26-27 compute similarity.
- Lines 30-34 convert to a percentage and add a small boost.

Current gap: import numpy as np is unused. Also, adding a fixed +15 boost can make scores feel nicer, but it reduces scientific purity. For portfolio clarity, explain it or replace with calibrated scoring.

Alternative embedding models:

- all-mpnet-base-v2: better quality, slower.
- OpenAI text-embedding models: strong hosted embeddings.
- BGE models: popular open-source retrieval embeddings.
- TF-IDF: simpler, keyword-based, weaker for semantic meaning.

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = model.encode(["Python APIs", "Backend Python REST API"])
score = cosine_similarity([embeddings[0]], [embeddings[1]])[0][0]
```

utils/extractor.py - structured resume extraction and URL title parsing

What it is: utils/extractor.py has two responsibilities. First, it defines ResumeData and extract_resume_details() for structured resume extraction. Second, it defines extract_real_title() to infer a job title from a job URL.

Why used here: Resume extraction turns raw resume text into fields such as skills, job titles, and years of experience. URL title extraction makes search result rows look more readable.

Real-world use cases: candidate profile parsing, HR automation, CRM data extraction, document information extraction.

Key components:

- ResumeData: Pydantic schema.
- extract_resume_details(): Gemini call with JSON schema.
- extract_real_title(): simple URL parsing for Naukri and LinkedIn.

Important lines:

- Lines 7-11 define expected resume fields.
- Lines 21-24 configure the Gemini model and system instruction.
- Lines 27-31 request JSON output matching ResumeData.
- Lines 34-40 call Gemini and validate JSON.
- Lines 42-71 parse job titles from known URL patterns.

Current gap: extract_resume_details() is not currently called by app.py. Connecting it is a strong next portfolio step.

Alternative extraction approaches:

- Regex: good for emails/phone numbers, weak for skills/experience.
- spaCy: good for named entity recognition.
- LLM structured output: flexible and strong, but needs validation.
- Dedicated resume parsing APIs: accurate, but paid and less educational.

```
from pydantic import BaseModel
from typing import List

class ResumeData(BaseModel):
    name: str
    skills: List[str]
    job_titles: List[str]
    total_years_experience: int
```

utils/pdf_reader.py - expected PDF reader module

What it is: app.py expects this file to provide `extract_text_from_pdf()` and `clean_text()`.

Current status: the source file is missing in the current workspace, although pycache files suggest it existed before. This is a critical gap because app.py imports it.

Expected purpose:

- `extract_text_from_pdf(path)`: open PDF and extract text from each page.
- `clean_text(text)`: normalize whitespace before sending text to scoring functions.

Alternative PDF tools:

- pypdf: simple PDF text extraction.
- pdfplumber: better for tables and layout.
- PyMuPDF: fast and strong for rendering/extraction.
- OCR with Tesseract: needed for scanned/image-only resumes.

```
from pypdf import PdfReader
import re

def extract_text_from_pdf(path):
    reader = PdfReader(path)
    text = ""
    for page in reader.pages:
        text += (page.extract_text() or "") + "\n"
    return text

def clean_text(text):
    return re.sub(r"\s+", " ", text).strip()
```

.env and .vscode/settings.json - environment and editor config

.env stores secrets such as `GOOGLE_API_KEY` and `APIFY_API_KEY`. Do not commit .env to GitHub.

.vscode/settings.json tells VS Code/PyLance to use the project virtual environment. This helps imports resolve correctly.

Important lines in settings.json:

- Line 2 points Pylance to `${workspaceFolder}/venv/bin/python`.
- Line 3 activates the environment in terminals.
- Lines 4-6 add the workspace root as an analysis path.

Alternative environment management:

- requirements.txt: simple dependency list.
- pyproject.toml: modern Python project config.
- Poetry/uv: stronger dependency locking.
- Conda: good for data science packages and system libraries.

```
# .env example, never commit real values
GOOGLE_API_KEY=your_key_here
APIFY_API_KEY=your_key_here
```

Exercises

- Open each file and write one sentence explaining its responsibility.
- Find one current code smell and explain why it matters.
- Restore or create `utils/pdf_reader.py` in a future coding step and test app.py import.

Interview questions

- Why did you split the app into modules?
- What does LangGraph add beyond a normal function call?
- Why combine local embedding scoring with LLM text generation?

5. Library, API, and Component Guide

This section explains the main libraries and APIs in beginner-friendly language.

Streamlit

What it is: a Python library for building web apps quickly.

Why used: it lets you create upload fields, buttons, tables, and downloads with Python.

Key functions: `st.title`, `st.file_uploader`, `st.text_input`, `st.button`, `st.spinner`, `st.session_state`, `st.dataframe`, `st.download_button`.

Real-world use cases: ML demos, dashboards, analytics tools.

Alternatives: Gradio for ML demos, Dash for dashboards, React/FastAPI for production flexibility.

Pandas

What it is: a data table library.

Why used: job records are converted to DataFrames and saved to CSV.

Key functions: `pd.DataFrame`, `pd.read_csv`, `pd.concat`, `drop_duplicates`, `to_csv`.

Real-world use cases: data cleaning, analytics, exports.

Alternatives: Polars for speed, SQLite/PostgreSQL for persistence, Spark for big data.

python-dotenv and os

What they are: `python-dotenv` loads variables from `.env`, and `os` reads them with `os.getenv()`.

Why used: secrets should not be hard-coded in source files.

Key functions: `load_dotenv()`, `os.getenv()`.

Real-world use cases: API keys, database URLs, feature flags.

Alternatives: cloud environment variables, secret managers, Docker secrets.

Pydantic

What it is: a data validation library.

Why used: it defines expected shapes such as `ResumeData` and `JobMatchReport`.

Key classes: `BaseModel`, `Field`.

Real-world use cases: API schemas, LLM structured output, config validation.

Alternatives: `dataclasses` for simple structures, `Marshmallow` for serialization, `TypedDict` for static hints only.

Google Gemini API

What it is: Google's LLM API.

Why used: it extracts resume details and generates fit summaries/cover letters.

Key classes/functions: `genai.configure`, `genai.GenerativeModel`, `ChatGoogleGenerativeAI`.

Real-world use cases: summarization, extraction, reasoning, content generation.

Alternatives: OpenAI, Anthropic, Mistral, local LLMs with Ollama.

Important note: `google.generativeai` is deprecated. The newer package is `google-genai`, which is already installed in the environment.

LangChain

What it is: a framework for composing prompts, models, parsers, and tools.

Why used: `matcher.py` builds a clear prompt -> LLM -> JSON parser chain.

Key components: `PromptTemplate`, `JsonOutputParser`, `LCEL` pipe operator.

Real-world use cases: AI chains, document Q&A, structured output, tool calling.

Alternatives: direct SDK calls, `LlamaIndex`, `Haystack`, `Instructor`.

LangGraph

What it is: a framework for stateful AI workflows.

Why used: agents/agent.py needs controlled steps and retry logic.

Key components: TypedDict state, StateGraph, nodes, conditional edges, END.

Real-world use cases: research agents, multi-step assistants, approval workflows.

Alternatives: plain Python, CrewAI, AutoGen, Prefect.

Apify Client

What it is: a Python client for Apify scraping actors.

Why used: searcher.py uses Apify's Google Search Scraper to collect job links/snippets.

Key functions/classes: ApifyClient, client.actor(...).call(), client.dataset(...).iterate_items().

Real-world use cases: scraping, market research, search result collection.

Alternatives: SerpAPI, Tavily, Bright Data, custom scrapers.

Sentence Transformers

What it is: a library for embedding text into numeric vectors.

Why used: local_scorer.py computes resume-job semantic similarity locally.

Key class/function: SentenceTransformer, model.encode().

Real-world use cases: semantic search, recommendations, clustering.

Alternatives: OpenAI embeddings, BGE embeddings, E5 embeddings, TF-IDF.

scikit-learn and NumPy

What they are: machine learning and numerical computing libraries.

Why used: cosine_similarity compares vectors. NumPy supports vector shapes and arrays.

Key function: sklearn.metrics.pairwise.cosine_similarity.

Real-world use cases: classification, clustering, similarity, metrics.

Alternatives: PyTorch, TensorFlow, SciPy, pure NumPy math.

pypdf

What it is: a PDF extraction library.

Why used: the expected pdf_reader.py module should extract resume text from uploaded PDF files.

Key class: PdfReader.

Real-world use cases: PDF parsing, document ingestion, text extraction.

Alternatives: pdfplumber, PyMuPDF, OCR tools.

Exercises

- For each library, write whether it is UI, data, API, AI, or storage related.
- Replace one library in theory and explain the tradeoff.
- Create a requirements.txt from the packages actually needed by the project.

Interview questions

- Why did you choose Streamlit for this stage?
- Why not use only an LLM for scoring?
- What is the risk of scraping job websites?

6. AI and ML Concepts

This project uses several practical AI concepts. Some are already implemented. Others are planned improvements that you should understand because they are common in AI interviews and real projects.

LLMs

What it is: A Large Language Model predicts and generates text based on patterns learned from huge datasets.

Why needed: The project needs natural language reasoning to summarize fit and write cover letters.

How it works internally: Text is split into tokens, converted into vectors, passed through transformer layers, and the model predicts likely next tokens.

Analogy: Like a highly trained autocomplete system that also learned patterns of reasoning and writing.

Practical example: Give it a resume and job snippet, ask it to produce a JSON summary.

Tokenization

What it is: The process of splitting text into small pieces called tokens.

Why needed: LLMs do not read raw characters the way humans do. They process token IDs.

How it works internally: Words, word parts, punctuation, and spaces are mapped to integer IDs.

Analogy: Breaking a paragraph into puzzle pieces before sending it to the model.

Practical example: A long resume plus job description may use many tokens, which affects cost and context limit.

Prompt Engineering

What it is: Designing instructions and context so an LLM returns useful output.

Why needed: A vague prompt gives vague answers. A structured prompt gives consistent fit summaries and cover letters.

How it works internally: The prompt becomes part of the token sequence that guides next-token prediction.

Analogy: Giving a junior assistant a clear task checklist instead of saying 'do something useful'.

Practical example: `matcher.py` tells the model it is an expert AI Career Coach and provides resume, job text, and JSON format instructions.

Structured Output

What it is: Asking the model to return data in a predictable format such as JSON.

Why needed: Apps need reliable fields, not random paragraphs.

How it works internally: The prompt/parser/schema constrains or validates model output.

Analogy: Asking someone to fill a form instead of writing a free-form essay.

Practical example: `JobMatchReport` requires `fit_analysis` and `cover_letter`.

Embeddings

What it is: Numeric vector representations of text.

Why needed: They let software compare meaning mathematically.

How it works internally: An embedding model maps text into a point in high-dimensional space. Similar meanings land near each other.

Analogy: Placing sentences on a map where similar sentences are close.

Practical example: 'Python backend APIs' and 'REST API developer using Python' should have nearby vectors.

Cosine Similarity

What it is: A math formula that compares the angle between two vectors.

Why needed: `local_scorer.py` uses it to compare resume and job embeddings.

How it works internally: If vectors point in similar directions, similarity is high. If unrelated, it is low.

Analogy: Two arrows pointing the same way mean the texts are aligned.

Practical example: Resume vector and job vector produce a decimal that becomes a match percentage.

Retrieval

What it is: Finding the most relevant information before answering.

Why needed: If the app stores many jobs, retrieval can find jobs most similar to a resume.

How it works internally: Query text becomes an embedding, then the system searches stored embeddings for nearest neighbors.

Analogy: Before answering a question, first pull the right pages from a book.

Practical example: Retrieve top 10 jobs from a job database that match the resume.

Vector Database

What it is: A database optimized to store and search embeddings.

Why needed: For many resumes/jobs, normal CSV search is not enough.

How it works internally: It indexes vectors using approximate nearest neighbor algorithms.

Analogy: A library catalog organized by meaning, not alphabet.

Practical example: Store job description embeddings in Chroma, Pinecone, Weaviate, Qdrant, or FAISS.

RAG

What it is: Retrieval-Augmented Generation. Retrieve relevant context, then ask an LLM to generate an answer using that context.

Why needed: It reduces hallucination and lets the model answer using your actual resume/job data.

How it works internally: User query -> retrieve relevant chunks -> insert chunks into prompt -> LLM generates answer.

Analogy: Open-book exam. The model answers after seeing the relevant notes.

Practical example: Retrieve the most relevant resume bullets and job requirements, then ask Gemini to explain gaps.

Agents

What it is: An AI workflow that can choose steps, call tools, and maintain state.

Why needed: Job hunting is not one step. It needs search, retry, score, summarize, and export.

How it works internally: A state object moves through nodes. Routing logic decides next node.

Analogy: A project manager moving a task through stages.

Practical example: LangGraph retries search with a broader role when the first search finds no jobs.

Temperature

What it is: A generation setting that controls randomness.

Why needed: Resume/job analysis should be consistent and factual.

How it works internally: Lower temperature makes the model choose more likely tokens. Higher temperature allows more variety.

Analogy: Low temperature is a careful checklist answer. High temperature is brainstorming.

Practical example: matcher.py uses temperature=0.2.

Common mistakes

- Using the word RAG when no retrieval step exists.
- Treating embedding similarity as perfect truth.
- Letting the LLM invent missing skills without evidence.

Exercises

- Explain embeddings to a non-technical friend.
- Create a tiny RAG design for storing 100 job descriptions.
- Compare cosine similarity and LLM reasoning in one paragraph.

Interview questions

- What is the difference between embeddings and LLMs?
- What problem does RAG solve?
- Why does token length matter?

- How does a vector database differ from a normal relational database?

7. Important Code Walkthroughs

This section gives line-by-line explanations for the most important code paths.

app.py startup and environment

Lines 1-6 import libraries and the compiled agent. Lines 13-15 load secrets from .env. This is good because API keys stay outside code. Lines 17-18 configure the Streamlit page and title.

```
load_dotenv()
gemini_key = os.getenv("GOOGLE_API_KEY")
serper_key = os.getenv("APIFY_API_KEY")
```

app.py resume upload

Line 32 creates the PDF uploader. Lines 34-37 save the uploaded PDF as temp_resume.pdf. Line 39 extracts text. Line 40 cleans the text. The cleaned text becomes the resume knowledge base for later scoring.

```
uploaded_file = st.file_uploader("Upload your Resume (PDF)", type=["pdf"])
if uploaded_file is not None:
    with open("temp_resume.pdf", "wb") as f:
        f.write(uploaded_file.getbuffer())
```

app.py agent invocation

Lines 71-80 create initial_state. This dictionary is the agent memory. It contains target role, location, resume text, keys, attempts, raw jobs, and scored jobs. Line 84 sends that state into LangGraph. The graph returns final_state.

```
initial_state = {
    "target_role": target_role,
    "location": search_location,
    "resume_text": st.session_state.cleaned_text,
    "gemini_key": gemini_key,
    "apify_key": serper_key,
    "search_attempts": 0,
    "raw_jobs": [],
    "scored_jobs": []
}
final_state = job_agent_app.invoke(initial_state)
```

agents/agent.py search node

search_for_jobs receives state, calls find_live_jobs, stores jobs in state['raw_jobs'], increments attempts, and returns state. In graph-based programming, every node receives and returns state.

```
jobs = find_live_jobs(
    state["target_role"],
    state["location"],
    state["apify_key"],
    max_results=5
)
```

agents/agent.py scoring node

score_found_jobs loops through every raw job. It uses the snippet as job_desc. It standardizes keys such as link, title, and company. It computes a local Match Score, then asks Gemini for Fit Summary and Cover Letter Draft.

```
math_score = calculate_local_match_score(state["resume_text"], job_desc)
job["Match Score"] = f"{math_score}%"
report = analyze_job_fit(state["resume_text"], job_desc, state["gemini_key"])
job["Fit Summary"] = report.fit_analysis
job["Cover Letter Draft"] = report.cover_letter
```

matcher.py chain

The chain is prompt | llm | parser. That means the input fills the prompt, the prompt goes to Gemini, and the Gemini output is parsed into JSON. This pattern is clean because every stage has one job.

```
parser = JsonOutputParser(pydantic_object=JobMatchReport)
chain = prompt | llm | parser
result = chain.invoke({"resume": resume_text, "job": job_desc})
```

local_scorer.py vector score

model.encode() turns text into vectors. reshape(1, -1) makes each vector a 2D array because cosine_similarity expects rows of vectors. raw_score is multiplied by 100 and rounded.

```
embeddings = model.encode([resume_text, job_desc])
resume_vector = embeddings[0].reshape(1, -1)
job_vector = embeddings[1].reshape(1, -1)
raw_score = cosine_similarity(resume_vector, job_vector)[0][0]
```

searcher.py CSV save

The function converts jobs into a DataFrame. If jobs_database.csv already exists, it reads existing rows, concatenates old and new, removes duplicates by link, saves the final table, and returns CSV bytes for the Streamlit download button.

```
new_df = pd.DataFrame(jobs_data)
combined_df = pd.concat([existing_df, new_df])
final_df = combined_df.drop_duplicates(subset=["link"], keep="first")
csv_for_download = final_df.to_csv(index=False).encode("utf-8")
```

Exercises

- Trace one resume upload from app.py to final DataFrame.
- Write comments for the scoring node in your own words.
- Change the mental model from 'functions' to 'state moving through graph nodes'.

Interview questions

- What does chain = prompt | llm | parser mean?
- Why does the graph return final_state?
- Why do we reshape vectors before cosine similarity?

8. Best Practices and Common Mistakes

These are the engineering practices that make the project stronger and safer.

Best practices

- Keep secrets in .env locally and deployment environment variables in production.
- Create requirements.txt or pyproject.toml so others can install dependencies.
- Restore missing source files instead of relying on pycache.
- Use structured output schemas for LLM responses.
- Validate all API responses before looping over them.
- Separate UI, search, scoring, model calls, and storage into modules.
- Use full job descriptions when possible, not only snippets.
- Add tests for title extraction, PDF cleaning, score function, and error routing.
- Show evidence for match score: matched skills, missing skills, weak evidence, and resume wording gaps.
- Respect website terms and user privacy when scraping and sending resume data to APIs.

Common mistakes in this project type

- Calling every AI app a RAG app even when there is no retrieval pipeline.
- Using only keyword overlap and missing semantic matches.
- Using only LLM judgment and getting inconsistent scores.
- Not handling scanned PDFs.
- Letting the app crash when an API returns an error string.
- Hard-coding API keys.
- Not explaining why a score was given.
- Forgetting that job snippets are incomplete job descriptions.

Recommended next implementation roadmap

Phase 1: Restore utils/pdf_reader.py and add requirements.txt.

Phase 2: Connect extract_resume_details() and show extracted profile.

Phase 3: Suggest target roles from extracted resume data.

Phase 4: Fetch full job descriptions from links.

Phase 5: Expand match report schema to include matched_skills, missing_skills, weak_evidence, missing_keywords, experience_gap, and resume_improvements.

Phase 6: Replace CSV with SQLite.

Phase 7: Add tests and README.

Phase 8: Deploy and record a demo video.

Exercises

- Turn the roadmap into GitHub issues.
- Write a test case for extract_real_title().
- Design a better MatchReport schema.

Interview questions

- What did you do to reduce hallucination?
- How do you validate LLM output?
- What would you improve first if given one week?

9. Portfolio Presentation Guide

A portfolio project should tell a story: problem, solution, architecture, demo, technical depth, limitations, and next steps.

README structure

Recommended README sections:

1. Project title and one-line summary.
2. Demo GIF or screenshots.
3. Problem statement.
4. Features.
5. Architecture diagram.
6. Tech stack.
7. How it works.
8. Setup instructions.
9. Environment variables.
10. Current limitations.
11. Future roadmap.
12. What I learned.

Features to highlight

- Resume upload and parsing.
- Autonomous job search workflow.
- Local semantic match scoring with embeddings.
- LLM-generated fit summaries and cover letters.
- CSV export and duplicate prevention.
- Planned structured gap analysis and role inference.

How to explain it in interviews

Use this short explanation:

I built an AI Job Hunt Copilot that accepts a resume, searches job listings, ranks them using local embedding similarity, and uses Gemini to explain fit and generate a cover letter. The app uses Streamlit for UI, Apify for search, LangGraph for workflow control, Sentence Transformers for local semantic scoring, LangChain for prompt/model/parser composition, and Pandas for CSV export. The next step is to fetch full job descriptions and store reports in SQLite.

Exercises

- Write your 30-second, 2-minute, and 5-minute project explanation.
- Create a screenshot checklist for the final README.
- Prepare one slide explaining the architecture.

Interview questions

- What tradeoffs did you make?
- How would you scale this app?
- What would you change for production?

10. Interview Questions and Practice Answers

Use these to prepare for portfolio discussions.

Beginner questions

Q: What is Streamlit?

A: Streamlit is a Python framework that turns Python scripts into interactive web apps.

Q: What is an embedding?

A: An embedding is a numeric vector that represents the meaning of text.

Q: Why use cosine similarity?

A: It measures how similar two embedding vectors are by comparing their direction.

Intermediate questions

Q: Why combine embeddings with an LLM?

A: Embeddings give fast, cheap similarity scores. The LLM gives human-readable reasoning and cover letters. Together they are more useful than either alone.

Q: What is LangGraph doing?

A: It controls a stateful workflow: search jobs, retry if needed, score jobs, then end.

Q: How do you avoid unreliable LLM output?

A: Use clear prompts, low temperature, structured output schemas, validation, and evidence from retrieved data.

Advanced questions

Q: How would you add RAG?

A: Store resume chunks and job description chunks as embeddings in a vector database. Retrieve the most relevant chunks for each job, then pass only those chunks to the LLM for gap analysis.

Q: How would you add a real database?

A: Start with SQLite tables for resumes, jobs, match_reports, and applications. For deployment and multi-user access, move to PostgreSQL or Supabase.

Q: How would you evaluate match score quality?

A: Create a small labeled dataset of resume/job pairs, compare model scores with human labels, measure rank correlation, and tune scoring weights.

Exercises

- Record yourself answering five questions.
- Write one technical deep dive about embeddings.
- Write one limitation and one improvement for each module.

11. Full Source Snapshot

This snapshot captures the current source files that the manual was based on. Keep this section as a learning reference.

app.py

```
1  from dotenv import load_dotenv
2  import streamlit as st
3  import pandas as pd
4  import os
5  import time
6  from agents.agent import job_agent_app
7
8  # Your custom modules
9  from utils.pdf_reader import extract_text_from_pdf, clean_text
10 from matcher import analyze_job_fit
11 from searcher import find_live_jobs, save_to_spreadsheet
12
13 load_dotenv()
14 gemini_key = os.getenv("GOOGLE_API_KEY")
15 serper_key = os.getenv("APIFY_API_KEY")
16
17 st.set_page_config(page_title="AI Job Search Agent", layout="wide")
18 st.title("[robot] AI Job Search Agent")
19
20 # --- SIDEBAR CONFIGURATION ---
21 # st.sidebar.header("API Configuration")
22 # gemini_key = st.sidebar.text_input("Enter Google Gemini API Key", type="password")
23 # serper_key = st.sidebar.text_input("Enter Serper.dev API Key", type="password")
24
25 # Session state to hold the resume text in memory
26 if "cleaned_text" not in st.session_state:
27     st.session_state.cleaned_text = ""
28
29 # --- PHASE 1: KNOWLEDGE BASE (RESUME) ---
30 st.subheader("Phase 1: Load Knowledge Base")
31 st.write("Upload your resume so the AI can use it to score jobs against your actual experience.")
32 uploaded_file = st.file_uploader("Upload your Resume (PDF)", type=["pdf"])
33
34 if uploaded_file is not None:
35     with st.spinner("Processing resume into memory..."):
36         with open("temp_resume.pdf", "wb") as f:
37             f.write(uploaded_file.getbuffer())
38
39         raw_text = extract_text_from_pdf("temp_resume.pdf")
40         st.session_state.cleaned_text = clean_text(raw_text)
41
42         st.success("Resume loaded successfully! You can now search for any role.")
43
44 # --- PHASE 2: SEARCH PARAMETERS ---
45 st.divider()
46 st.subheader("Phase 2: Define Your Target")
47 st.info("What kind of roles are we hunting for today?")
48
49 col1, col2 = st.columns(2)
50 with col1:
51     target_role = st.text_input("Target Job Title", placeholder="e.g., Unity Developer, ML Engineer")
52 with col2:
53     search_location = st.text_input("Target Location", placeholder="e.g., Remote, India")
54
55 # --- PHASE 3: AUTONOMOUS JOB HUNTER ---
56 st.divider()
57 st.subheader("Phase 3: Autonomous Job Agent")
58
59 if st.button("Deploy Agent"):
60     # 1. Safety Checks
61     if not serper_key or not gemini_key: # (Or your apify/rapidapi key variable)
62         st.error("Please provide your API Keys in the sidebar.")
63     elif not target_role:
64         st.warning("Please enter a target job title to begin searching.")
65     elif not st.session_state.cleaned_text:
66         st.warning("Please upload your resume in Phase 1 first!")
67     else:
68         with st.spinner(f"[launch] Deploying Autonomous Agent to hunt for '{target_role}'..."):
69
70             # 2. Define the Agent's Starting Memory (State)
71             initial_state = {
72                 "target_role": target_role,
73                 "location": search_location,
74                 "resume_text": st.session_state.cleaned_text,
75                 "gemini_key": gemini_key,
76                 "apify_key": serper_key, # Pass whatever key you are using for scraping
77                 "search_attempts": 0,
```

```

78         "raw_jobs": [],
79         "scored_jobs": []
80     }
81
82     # 3. WAKE UP THE AGENT!
83     # The agent will now search, expand the query if needed, and score the jobs
autonomously.
84     final_state = job_agent_app.invoke(initial_state)
85
86     # 4. Extract the final results from the agent's memory
87     scored_jobs = final_state.get("scored_jobs", [])
88     final_search_term = final_state.get("target_role")
89
90     # 5. Display the Results
91     if scored_jobs:
92         st.success(f"Agent finished! It successfully evaluated {len(scored_jobs)}
jobs.")
93
94         # If the agent had to expand the query, let the user know!
95         if final_search_term != target_role:
96             st.info(f"[retry] The agent couldn't find enough jobs for '{target_role}',
so it autonomously expanded the search to: '{final_search_term}'")
97
98         # Save to CSV
99         csv_file, new_count = save_to_spreadsheet(scored_jobs)
100
101         if csv_file is not None:
102             st.download_button(
103                 label="[download] Download Your Scored Jobs & Cover Letters (CSV)",
104                 data=csv_file,
105                 file_name="agent_tracked_jobs.csv",
106                 mime="text/csv"
107             )
108
109         # Display cleanly in Streamlit
110         df = pd.DataFrame(scored_jobs)
111         cols = ["title", "company", "Match Score", "location", "link", "Fit Summary"]
112         df_display = df[[c for c in cols if c in df.columns]]
113         st.dataframe(df_display, use_container_width=True)
114
115     else:
116         st.error("The agent completed its workflow but couldn't find any valid jobs
to score after multiple attempts.")
117
118

```

agents/agent.py

```

1  from typing import TypedDict, List
2  from langgraph.graph import StateGraph, END
3  import streamlit as st
4  from utils.extractor import extract_real_title
5  # Import your existing tools
6  from searcher import find_live_jobs
7  from matcher import analyze_job_fit
8  from local_scorer import calculate_local_match_score
9
10 # 1. Define the Agent's Memory (State)
11 class JobAgentState(TypedDict):
12     target_role: str
13     location: str
14     resume_text: str
15     gemini_key: str
16     apify_key: str # Or apify_key depending on what you used
17     search_attempts: int
18     raw_jobs: List[dict]
19     scored_jobs: List[dict]
20
21 # 2. Define the Nodes (Actions)
22
23 def search_for_jobs(state: JobAgentState):
24     """Hits the API to find jobs based on the current target_role."""
25     print(f"[search] Searching for: {state['target_role']}...")
26
27     jobs = find_live_jobs(
28         state["target_role"],
29         state["location"],
30         state["apify_key"],
31         max_results=5
32     )
33
34     # Update the state memory
35     if jobs != "API_ERROR" and jobs:
36         state["raw_jobs"] = jobs
37     else:
38         state["raw_jobs"] = []
39
40     state["search_attempts"] += 1
41     return state
42

```



```

43 def expand_search_query(state: JobAgentState):
44     """If no jobs are found, the AI rewrites the search query to try again."""
45     print("[retry] Not enough jobs found. Expanding search query...")
46
47     # In a full production app, you would use an LLM here to generate synonyms.
48     # For now, we will do a simple fallback logic to demonstrate the graph loop.
49     old_role = state["target_role"]
50     state["target_role"] = f"{old_role} OR Software Engineer"
51
52     return state
53
54 def score_found_jobs(state: JobAgentState):
55     """Runs the LangChain matcher on the successfully found jobs."""
56     print("[score] Scoring jobs...")
57     scored = []
58
59     for job in state["raw_jobs"]:
60         job_desc = job.get("Snippet", "")
61         # 1. Pull data using the EXACT keys from the Apify scraper
62         job_desc = job.get("Snippet", "")
63         job_link = job.get("Application Link", "No link provided")
64         fallback_title = job.get("Job Title/Search Term", "Unknown Role")
65
66         # 2. Reconstruct the real title from the URL
67         real_title = extract_real_title(job_link, fallback_title)
68
69         # 3. Standardize the dictionary keys so Pandas/Streamlit don't crash
70         job["link"] = job_link
71         job["title"] = real_title
72         job["company"] = "Company Not Listed"
73         # =====
74
75         if job_desc:
76             # --- Phase 3 Local Vector Scoring ---
77             math_score = calculate_local_match_score(state["resume_text"], job_desc)
78             job["Match Score"] = f"{math_score}%"
79
80             # --- Phase 1: LangChain (Fit Summary & Cover Letter only) ---
81             report = analyze_job_fit(state["resume_text"], job_desc, state["gemini_key"])
82             job["Fit Summary"] = report.fit_analysis
83             job["Cover Letter Draft"] = report.cover_letter
84         else:
85             job["Match Score"] = "N/A"
86
87         scored.append(job)
88
89     state["scored_jobs"] = scored
90     return state
91
92 # 3. Define the Routing Logic
93 def route_after_search(state: JobAgentState):
94     """Decides if we should score the jobs, or try searching again."""
95
96     # If we found jobs, proceed to scoring
97     if len(state["raw_jobs"]) > 0:
98         return "score_jobs"
99
100     # If we found 0 jobs, but we haven't tried 3 times yet, try expanding the query
101     elif state["search_attempts"] < 3:
102         return "expand_query"
103
104     # If we failed 3 times, give up and end the graph
105     else:
106         return END
107
108 # 4. Build and Compile the Graph
109 workflow = StateGraph(JobAgentState)
110
111 # Add the nodes
112 workflow.add_node("search_jobs", search_for_jobs)
113 workflow.add_node("expand_query", expand_search_query)
114 workflow.add_node("score_jobs", score_found_jobs)
115
116 # Define the starting point
117 workflow.set_entry_point("search_jobs")
118
119 # Add the conditional edges (The Brain)
120 workflow.add_conditional_edges(
121     "search_jobs",
122     route_after_search,
123     {
124         "score_jobs": "score_jobs",
125         "expand_query": "expand_query",
126     },
127     END: END
128 )
129
130 # If it expands the query, it MUST go back to search again
131 workflow.add_edge("expand_query", "search_jobs")
132
133 # If it scores the jobs, the workflow is finished

```

```

134 workflow.add_edge("score_jobs", END)
135
136 # Compile it into a runnable application!
137 job_agent_app = workflow.compile()

```

matcher.py

```

1 from langchain_google_genai import ChatGoogleGenerativeAI
2 from langchain_core.prompts import PromptTemplate
3 from langchain_core.output_parsers import JsonOutputParser
4 from pydantic import BaseModel, Field
5
6 # --- 1. THE BLUEPRINT (Structured Output) ---
7 # We define exactly what we want back. LangChain handles forcing the LLM to output this exact
JSON format.
8 class JobMatchReport(BaseModel):
9     fit_analysis: str = Field(description="A 2-sentence summary of why they are a good fit or
what they are missing.")
10     cover_letter: str = Field(description="A short, 3-paragraph cover letter tailored
specifically to this company and role.")
11
12 def analyze_job_fit(resume_text, job_desc, gemini_key):
13     # --- 2. INITIALIZE THE LLM ---
14     # We set temperature to 0.2 so the AI is highly factual and doesn't hallucinate skills.
15     llm = ChatGoogleGenerativeAI(
16         model="gemini-2.5-flash",
17         google_api_key=gemini_key,
18         temperature=0.2
19     )
20
21     # --- 3. SET UP THE PARSER ---
22     # We tell LangChain to use our blueprint from Step 1
23     parser = JsonOutputParser(pydantic_object=JobMatchReport)
24
25     # --- 4. CREATE THE PROMPT TEMPLATE ---
26     prompt = PromptTemplate(
27         template="""You are an expert AI Career Coach and Recruiter.
28         Analyze the candidate's Resume against the Target Job Description.
29
30         Resume KNOWLEDGE BASE:
31         {resume}
32
33         TARGET JOB DESCRIPTION:
34         {job}
35
36         {format_instructions}
37         """,
38         input_variables=["resume", "job"],
39         # This magically injects the JSON rules into the prompt so the LLM knows what to do
40         partial_variables={"format_instructions": parser.get_format_instructions()}
41     )
42
43     # --- 5. BUILD THE CHAIN ---
44     # This is the magic of LangChain. Data flows left to right:
45     # Prompt is filled -> sent to LLM -> output is parsed into our JSON Blueprint
46     chain = prompt | llm | parser
47
48     # --- 6. EXECUTE ---
49     try:
50         # We pass in our variables and trigger the chain
51         result = chain.invoke({"resume": resume_text, "job": job_desc})
52
53         # We wrap the dictionary in a simple class so it works seamlessly
54         # with your existing app.py code (report.match_percentage, etc.)
55         class Report:
56             def __init__(self, data):
57                 self.fit_analysis = data["fit_analysis"]
58                 self.cover_letter = data["cover_letter"]
59
60             return Report(result)
61     except Exception as e:
62         print(f"LangChain Error: {e}")
63         # Return empty/fallback values if something goes wrong
64         class ErrorReport:
65             match_percentage = 0
66             fit_analysis = "Error analyzing fit."
67             cover_letter = "Error generating letter."
68         return ErrorReport()

```

searcher.py

```

1 from apify_client import ApifyClient
2 import pandas as pd
3 import os
4
5
6 def find_live_jobs(job_title: str, location: str, apify_api_key: str, max_results: int = 10):
7     """
8     Commands the OFFICIAL Apify Cloud Actor to execute a Google search and returns

```

```

9     the results, safely extraction fields from the Run object instance.
10    """
11    # Removed the strict trailing slashes so Apify's internal filter doesn't panic
12    search_query = (
13        f'{job_title} {location} '
14        f'(site:naukri.com/job-listings '
15        f'OR site:indeed.com/viewjob '
16        f'OR site:linkedin.com/jobs/view)')
17
18    client = ApifyClient(apify_api_key)
19
20    run_input = {
21        "queries": search_query,
22        "maxPagesPerQuery": 1,
23        "resultsPerPage": max_results
24    }
25
26    jobs_found = []
27
28    try:
29        # 1. Call the official Apify actor
30        run = client.actor("apify/google-search-scraper").call(run_input=run_input)
31
32        # 2. Extract the dataset ID safely whether 'run' is an object or a dict
33        if isinstance(run, dict):
34            dataset_id = run.get("defaultDatasetId")
35        else:
36            # It's a 'Run' object instance, use attribute access
37            dataset_id = getattr(run, "default_dataset_id", None) or getattr(run,
38            "defaultDatasetId", None)
39
40            if not dataset_id:
41                return "ERROR: Could not locate defaultDatasetId from Apify run configuration."
42
43        # 3. Fetch the results from the Actor's cloud dataset
44        dataset_items = client.dataset(dataset_id).iterate_items()
45
46        for page_data in dataset_items:
47            # Extract organic results array from the page payload
48            organic_results = page_data.get("organicResults", [])
49
50            for result in organic_results:
51                link = result.get("url", "")
52                snippet = result.get("description", "No description available.")
53
54                if link:
55                    jobs_found.append({
56                        "Job Title/Search Term": job_title,
57                        "Snippet": snippet,
58                        "Application Link": link,
59                        "Status": "Pending Review"
60                    })
61
62    except Exception as e:
63        print(f"[ERROR] Apify Gateway failed: {str(e)}")
64        return f"ERROR: {str(e)}"
65
66    return jobs_found
67
68 def save_to_spreadsheet(jobs_data: list, filename: str = "jobs_database.csv"):
69     # 1. Safety check for empty or error data
70     if not jobs_data or (isinstance(jobs_data, str) and jobs_data.startswith("ERROR:")):
71         return None, 0
72
73     # 2. Convert new jobs to a DataFrame
74     new_df = pd.DataFrame(jobs_data)
75
76     # 3. Check if we have historical data saved on the server
77     if os.path.exists(filename):
78         existing_df = pd.read_csv(filename)
79         # Combine old and new data
80         combined_df = pd.concat([existing_df, new_df])
81         # Drop duplicates using the unique job link
82         final_df = combined_df.drop_duplicates(subset=['link'], keep='first')
83
84         # Calculate how many genuinely new jobs we found
85         new_jobs_added = len(final_df) - len(existing_df)
86     else:
87         # First time running
88         final_df = new_df
89         new_jobs_added = len(final_df)
90
91     # 4. Save to the server so it remembers for next time
92     final_df.to_csv(filename, index=False)
93
94     # 5. USER PERSPECTIVE: Convert the final table into raw CSV text format
95     # The .encode('utf-8') turns it into a format Streamlit can send as a download
96     csv_for_download = final_df.to_csv(index=False).encode('utf-8')
97
98     return csv_for_download, new_jobs_added

```

local_scorer.py

```
1 from sentence_transformers import SentenceTransformer
2 from sklearn.metrics.pairwise import cosine_similarity
3 import numpy as np
4
5 # 1. Load a lightweight, super-fast embedding model directly to your machine
6 # This model converts sentences into 384-dimensional math vectors
7 model = SentenceTransformer('all-MiniLM-L6-v2')
8
9 def calculate_local_match_score(resume_text: str, job_desc: str) -> int:
10     """
11     Converts text to vectors and calculates the mathematical Cosine Similarity.
12     Returns a score out of 100.
13     """
14     try:
15         # 2. Convert the texts into numbers (Embeddings)
16         # We put them in a list so the model processes them together
17         embeddings = model.encode([resume_text, job_desc])
18
19         # embeddings[0] is the resume vector
20         # embeddings[1] is the job description vector
21         resume_vector = embeddings[0].reshape(1, -1)
22         job_vector = embeddings[1].reshape(1, -1)
23
24         # 3. Calculate the Cosine Similarity (The Dot Product math!)
25         # This returns a decimal between 0 and 1 (e.g., 0.85)
26         similarity_matrix = cosine_similarity(resume_vector, job_vector)
27         raw_score = similarity_matrix[0][0]
28
29         # 4. Convert the decimal to a nice clean percentage (e.g., 85)
30         percentage_score = int(round(raw_score * 100))
31
32         # A little buffer: vector matching can be harsh, so we can boost it slightly
33         # just to make the scores feel more human-readable.
34         final_score = min(100, percentage_score + 15)
35
36         return final_score
37
38     except Exception as e:
39         print(f"Vector Math Error: {e}")
40         return 0
```

utils/extractor.py

```
1 from pydantic import BaseModel
2 from typing import List
3 import google.generativeai as genai
4 from google.generativeai import types
5
6 # 1. We keep our same Blueprint (Schema)
7 class ResumeData(BaseModel):
8     name: str
9     skills: List[str]
10     job_titles: List[str]
11     total_years_experience: int
12
13 def extract_resume_details(resume_text: str, api_key: str) -> ResumeData:
14     """
15     Sends the cleaned resume text to Gemini and forces it to return
16     the data formatted exactly like our ResumeData blueprint.
17     """
18     # Initialize the modern Gemini Client
19     client = genai.configure(api_key=api_key)
20
21     model = genai.GenerativeModel(
22         model_name='gemini-2.5-flash',
23         system_instruction="You are an expert HR assistant. Read the resume text and extract
the required fields accurately."
24     )
25
26     # Define the configuration to enforce our Pydantic schema
27     config = types.GenerationConfig(
28         response_mime_type="application/json",
29         response_schema=ResumeData,
30         temperature=0.1 # Low temperature makes the AI more deterministic and accurate
31     )
32
33     # Call the lightweight, ultra-fast Gemini 2.5 Flash model
34     response = model.generate_content(
35         contents=resume_text,
36         generation_config=config,
37     )
38     # Gemini returns a clean JSON string in response.text.
39     # We parse it directly back into our Pydantic structure.
40     return ResumeData.model_validate_json(response.text)
41
42 def extract_real_title(job_link, fallback_title):
43     """Attempts to extract the real job title from the URL based on the platform."""
```

```

44     try:
45         if "naukri.com/job-listings-" in job_link:
46             url_path = job_link.split("job-listings-")[1].split("-")
47             clean_words = []
48             for word in url_path:
49                 if word.isdigit():
50                     break
51                 clean_words.append(word)
52             return " ".join(clean_words[:4]).title()
53
54         elif "linkedin.com/jobs/view/" in job_link:
55             # LinkedIn format: linkedin.com/jobs/view/unity-developer-at-company-12345
56             url_path = job_link.split("jobs/view/")[1].split("?")[0].split("-")
57             clean_words = []
58             for word in url_path:
59                 # Stop if we hit a number or the word "at" (which precedes the company name)
60                 if word.isdigit() or word.lower() == "at":
61                     break
62                 clean_words.append(word)
63             return " ".join(clean_words[:4]).title()
64
65         else:
66             # For Indeed or any unknown portal, we safely use the fallback
67             return fallback_title.title()
68
69     except Exception:
70         # If any parsing fails, never crash. Just return the fallback.
71         return fallback_title.title()

```

utils/pdf_reader.py

[Missing file in current workspace: utils/pdf_reader.py]

.vscode/settings.json

```

1  {
2      "python.defaultInterpreterPath": "${workspaceFolder}/venv/bin/python",
3      "python.terminal.activateEnvironment": true,
4      "python.analysis.extraPaths": [
5          "${workspaceFolder}"
6      ]
7  }

```